

Valeur–Maselli Marius
Khalfouni Fady
Meziani Fahim
Coquin Arnaud

Rapport Final SAE 3.1

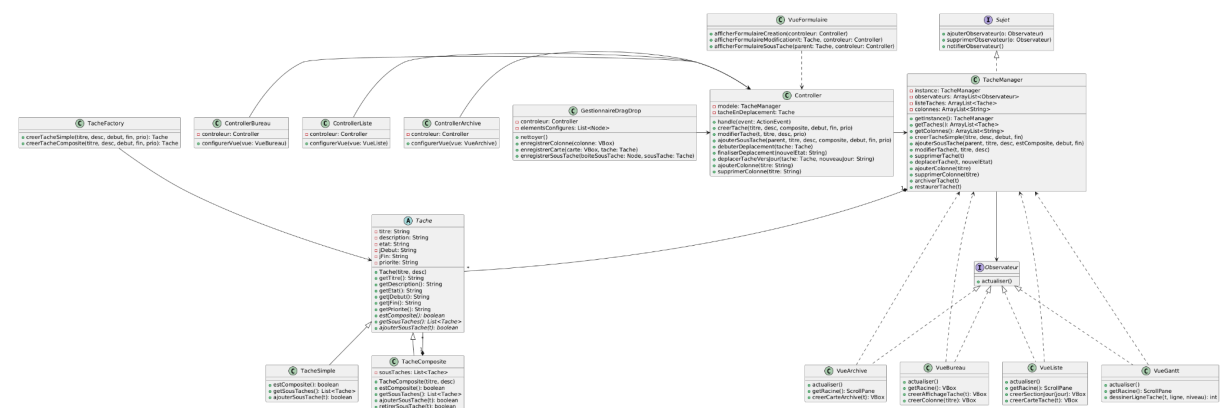
Sommaire :

1. Liste des fonctionnalités
2. Diagramme de classe finale
3. Répartition du travail
4. Eléments originaux
5. Eléments modifiés par rapport à l'étude préalable
6. Patrons de conceptions et d'architectures utilisés
7. Graphe de scène
8. Mode d'emploi

Liste des fonctionnalités réalisées :

- Créer une tâche simple
- Modifier / supprimer une tâche
- Liste de sous-tâches (création, affichage)
- Ajouter les colonnes (colonne de base)
- Visualiser la description
- Afficher les tâches dans les colonnes
- Drag&drop dans les colonnes
- Sérialisation
- Afficher la vue liste
- Afficher la vue Gantt
- Créer / supprimer des colonnes
- Gérer les sous-tâches de sous-tâches
- Modifier les sous-tâches
- Afficher la vue Archive
- Gérer l'archivage des tâches
- Gérer la priorité des tâches
- Drag&drop sur les sous-tâches
- Système de date
- Restauration des tâches

Diagramme de classe finale



Répartition du travail

Arnaud :

- Patron Observateur
- Patron Singleton
- MVC v1
- Modifier / supprimer une tâche mère
- Visualiser la description dans la vue Bureau
- Afficher la vue Gantt
- Gérer les priorités
- Système de dates

Fahim :

- Patron Fabrique
- Créer une tâche mère
- Afficher la liste des tâches dans une vue (bureau et texte)
- Ajouter les colonnes initiales, les tâches vont dans la colonne de gauche à la création
- MVC v2
- Drag&drop des tâches mères
- Créer / supprimer des colonnes
- Gérer les sous-tâches de sous-tâches
- Modifier les sous-tâches
- Afficher la vue Archive
- Gérer l'archivage des tâches

Marius :

- Patron Composite
- Créer les sous-tâches
- MVC v3, version correcte
- Sérialisation
- Afficher la vue Liste
- Drag&drop des sous-tâches

Fady :

- Tests unitaires
- Diagrammes

Éléments originaux

Arnaud :

Ma fierté principale dans ce projet est le développement de la classe `VueGantt`, un élément qui assure une visualisation temporelle des tâches. Cela repose sur une structure récursive avec la méthode `afficherTacheRecuratif`, permettant de parcourir l'arborescence des tâches composites tout en adaptant automatiquement les couleurs et l'indentation selon le niveau de profondeur. La gestion du temps est également automatisée : le diagramme calcule le décalage nécessaire pour les tâches s'étendant sur plusieurs semaines et ajuste le nombre de colonnes du `GridPane`. Bien que j'ai conscience que l'utilisation de la classe `Line` aurait été idéale pour relier physiquement les tâches mères à leurs enfants, la complexité des lignes entre les cellules de la grille ne m'a pas permis d'obtenir un rendu satisfaisant. J'ai donc préféré ne pas l'implémenter.

Fahim :

Ma fierté principale dans ce projet est le développement du système d'archivage/suppression et de la vue `VueArchive`. Cette fonctionnalité repose sur un algorithme de recherche récursif, `chercherArchivesRecuratif`, capable de traverser intégralement l'arbre du patron `Composite` pour extraire les tâches archivées, qu'elles soient des tâches mères ou des sous-tâches. Cela m'a permis d'utiliser ce que j'ai appris dans le cours d'algo en situation réelle. La restauration est également gérée grâce à la mémorisation de l'`ancienEtat` une variable string qui est sérialisée, qui fait en sorte que quand on restaure ça soit fluide en indiquant leur provenance textuelle pour garder le contexte sans alourdir l'interface. J'ai également géré des cas : par exemple, si l'on restaure une tâche dont la colonne d'origine a été supprimée, la tâche récupère correctement son état dans le modèle (la donnée est sauvegardée), et il suffit de recréer la colonne pour qu'elle réapparaisse visuellement. Enfin, la restauration d'une tâche mère ou d'une sous-tâche qui ont contenu d'autres tâches préserve l'intégralité de ses sous-tâches, garantissant qu'aucune donnée hiérarchique n'est perdue lors du processus.

Pour l'affichage d'un grand nombre de tâches dans la vue archive, j'ai réalisé qu'un `borderpane` ne suffirait pas : sa risque de dépasser de la fenêtre. J'ai donc anticipé le besoin et cherché comment faire pour pouvoir scroller si ça dépasse de la fenêtre. En effectuant des recherches j'ai découvert découvrir et compris le fonctionnement de la classe `ScrollPane`. j'ai donc pu l'intégrer pour que l'utilisateur puisse toujours accéder à l'ensemble de son archive.

Marius :

Ce dont je suis le plus fier dans ce projet est d'avoir réussi à implémenter un système de drag & drop qui permet aux sous-tâches d'être déplacées n'importe où, aussi bien dans la vue Bureau que dans la vue Liste, en respectant complètement la hiérarchie du patron Composite. Au départ, cette fonctionnalité semblait simple mais s'est avérée complexe à réaliser proprement. J'ai d'abord constaté que le code original mélangeait tout dans le Main, alors j'ai décidé de créer des contrôleurs spécialisés pour chaque vue, ce qui a déjà clarifié l'architecture. Cependant, le vrai défi était de faire en sorte que le programme comprenne la différence entre une tâche principale et une sous-tâche pendant le drag & drop. Pour cela, j'ai développé la méthode `estSousTache()` dans `TacheManager` qui vérifie si une tâche est dans la liste principale ou cachée dans une hiérarchie. Cette détection permet ensuite d'adapter le comportement : quand une sous-tâche est déposée sur une nouvelle colonne dans la vue Bureau, elle est extraite pour devenir indépendante avec `extraireVersColonne()`, alors qu'une tâche normale change juste d'état. Même logique dans la vue Liste avec `extraireVersJour()` pour les sous-tâches. Ce qui me rend fier, c'est d'avoir pensé à tous les cas particuliers : éviter qu'une tâche devienne sous-tâche d'elle-même, conserver la durée des tâches déplacées, et surtout permettre aux utilisateurs de manipuler librement la hiérarchie des tâches de manière intuitive, tout en gardant une architecture propre où chaque classe a un rôle bien défini.

Fady :

Éléments modifiés par rapport à l'étude préalable

Par rapport à l'étude préalable, nous avons décidé de ne pas implémenter le patron de conception Stratégie car nous avons estimé qu'il n'était pas nécessaire au bon fonctionnement de l'application.

Le visuel de l'application n'est plus le même.

La vue paramètre qui permettait de modifier la couleur du fond ou autre modification n'a pas été implémentée.

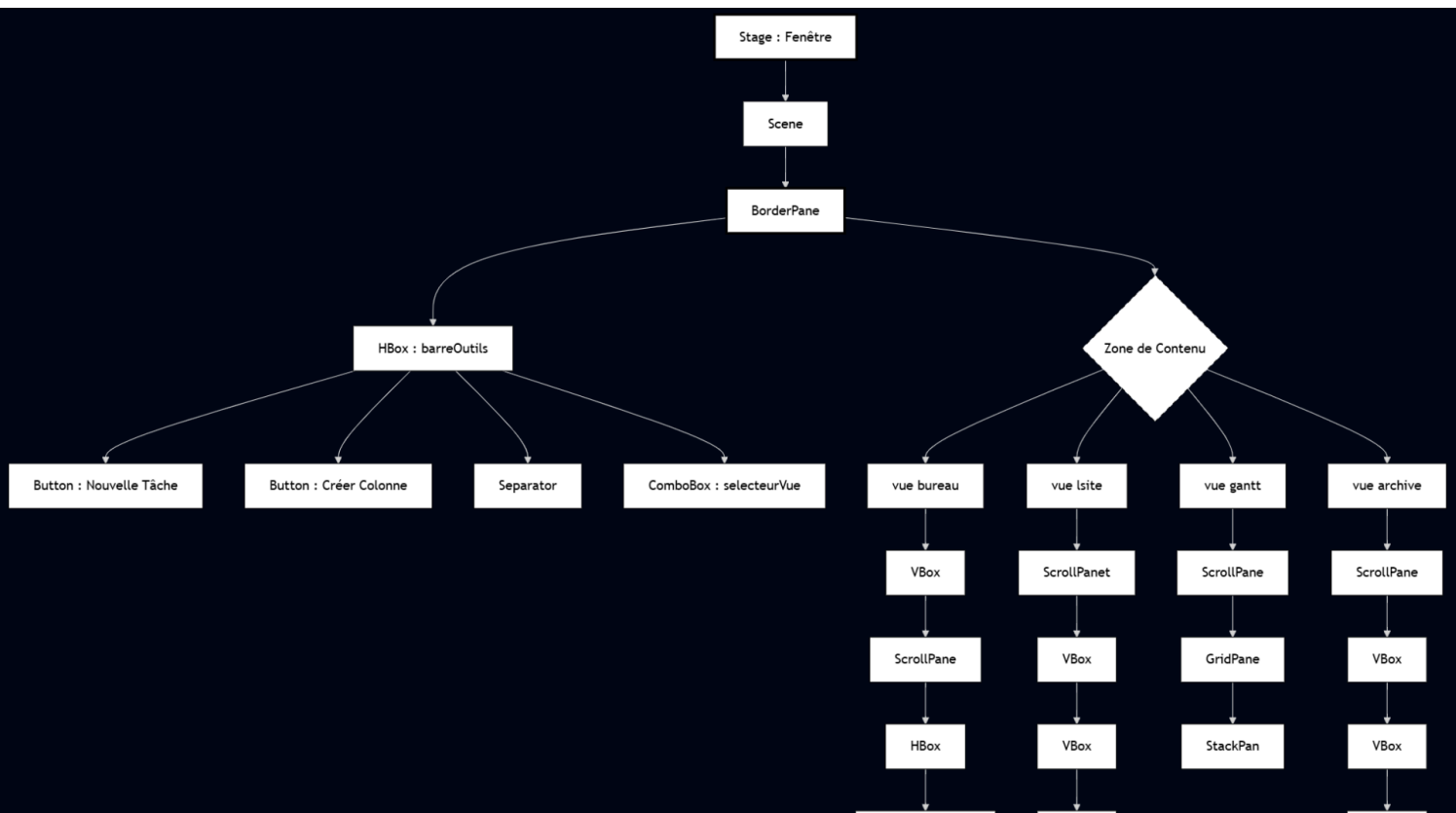
La vue gantt et système de date a été fait par jour alors que nous avions prévu de l'implémenter par date (JJ/MM/AAAA).

Nous n'avons pas gérer le système de dépendances entre 2 tâches et nous n'avons pas calculé l'avancement d'une tâche.

Patrons de conceptions et d'architectures utilisés

Nous avons utilisé le patron d'architecture MVC afin d'avoir une application visuelle avec un modèle des vues et un contrôleur. Pour les patrons de conceptions, nous avons utilisé le patron Observateur afin de pouvoir implémenter le patron MVC et donc que les vues se mettent à jour, le patron Singleton qui permet d'avoir une seule instance de notre application, le patron Fabrique afin de créer soit des tâches simples, soit des tâches composites, et enfin le patron Composite afin de créer des sous-tâches et des tâches mères.

Graphes de scène



Mode d'emploi

1. Prérequis Techniques

- Java Development Kit (JDK) : Version 23 ou supérieure recommandée
- Avoir IntelliJ IDEA d'installer ou visual studio code avec les bonnes extensions d'installation.

2. Organisation des Fichiers

L'architecture du projet pour les classes de l'application :

- SAE.S3.01_meziani_coquin_valeur-maseilli_khalfouni\sae
 - src/main/java
 - source/ (Package principal)
 - Main.java (Point d'entrée)
 - Controller.java
 - TacheManager.java
 - ... (et toutes les autres classes .java)
 - taches.sauvegarde (Généré automatiquement après le premier lancement)
 - colonnes.sauvegarde (Généré automatiquement)

L'architecture du projet pour les classes tests de l'application :

- SAE.S3.01_meziani_coquin_valeur-maseilli_khalfouni\sae
 - src/main/java
 - test/
 - TacheManagerTest.java
 - TestTache.java
 - TestTacheAvance.java

3. Importation et Configuration dans IntelliJ

1. Création du projet :
 - cloner le fichier depuis le git (clé ssh : git@github.com:Fhm11/SAE.S3.01_meziani_coquin_valeur-maseilli_khalfouni.git)
 - lancer intelliJ
 - ouvrir le projet
 - setup le sdk si besoin (normalement non)
2. Configuration des dépendances (JavaFX) :
 - Allez dans File > Project Structure > Libraries.
 - Cliquez sur le + (Java) et sélectionnez le dossier lib de votre SDK JavaFX téléchargé.
 - Validez pour ajouter la bibliothèque au projet.
 - Allez dans File > Project Structure > Libraries.
 - Cliquez sur le + (Java) et sélectionnez from maven
 - rentre: org.junit.jupiter:junit-jupiter:5.10.0
 - "ok" puis appliquer

4. Lancement de l'Application

1. Ouvrez le fichier source/Main.java dans l'éditeur.
2. Recherchez la méthode main ou la classe Main.
3. Cliquez sur la flèche verte de lecture (Run).
4. L'application doit s'ouvrir sur la Vue Bureau par défaut.

5. Accès aux Fonctionnalités Principales

Une fois l'application lancée, voici comment accéder aux fonctionnalités clés pour les tester :

- Création de Tâche : Cliquez sur le bouton vert "Nouvelle Tâche".
- Gestion des Colonnes :
 - Ajouter : Bouton "Créer Colonne" dans la barre d'outils.
 - Supprimer : Cliquez sur la croix rouge à droite du titre d'une colonne.
- Navigation entre les Vues : Utilisez le menu déroulant en haut pour basculer entre :
 - *Vue Bureau.*
 - *Vue Liste par Jour.*
 - *Vue Gantt.*
 - *Vue Archive.*
- Manipulation des Tâches :
 - Déplacement : Glissez-déposez une tâche d'une colonne à une autre.
 - Sous-tâche : Glissez une tâche sur une autre tâche composite pour en faire une sous-tâche une sous tâche peut devenir une tâche mere.
 - Archivage : Cliquez sur le bouton "Archiver" présent sur chaque carte.
 - Restauration : Depuis la *Vue Archive*, cliquez sur "Restaurer" pour remettre une tâche dans son état d'origine ou supprimer pour la supprimer.

6. Point important à savoir

Une sous-tâche doit forcément être datée dans le même intervalle qu'une tâche mère.

Si vous supprimez une colonne les tâches et sous-tâches sont supprimées.

Une tâche ou sous-tâche archiver ou la colonne n' existe plus peut être restaurée il suffira juste de recréer la colonne avec le nom à l'identique (majuscule compris).

Les colonnes ne sont pas déplaçables.

La vue Gantt dure seulement 2 semaines consécutives.

Prévoir à l'avance si une tâche pourra contenir des sous-tâches.

Pour déplacer une tâche ou sous-tâches maintenir un endroit sur tout la ligne.

Pour modifier une tâche ou sous tâche il suffit de double cliquer sur celle ci

Si une erreur est presenter une popup s'ouvrira pour vous l' indiquez

Pas besoin de cliquer sur un bouton sauvegarder, des qu on quitte l application tout se sauvegarde automatiquement et quand on relance le main le fichier chargé et vous retrouvez toutes vos données

Si vous souhaitez repartir à 0 niveau données supprimer les fichier colonnes.sauvegardes et taches.sauvegardes puis relancer le main

Pour transformer une tâche principale en sous-tâche d'une autre, glissez la tâche "enfant" et déposez-la directement sur la tâche mere.